

Software Best Practices - Personal Experience

In industry it is uncommon for an engineer to be presented with a well-scoped problem. The problems which one encounters in this environment lack clear ownership, are not well understood, and are often technically complex. In order to deliver value in this environment one should adhere to the following key practices: centralize requirements, test-driven development, and defensive coding.

Centralize Requirements

All software exists with a purpose. The clarity and scope of the software are key to how effectively it can be delivered. To ensure the scope is well defined, make use of a single central document. This document should be publicized and reviewed by all stakeholders. The document will change as the requirements change, this is inevitable, but returning to the same document allows for these changes to be tracked clearly. Changes to this document should be reviewed by all stakeholders.

Test Driven Development

The most powerful tool from a software engineering standpoint is test driven development. In a dynamic environment the requirements will change and the code will need to adapt. If the engineer structures their code to enable test-driven development, the overhead of making and testing adaptations will be trivial.

Defensive Coding

Software will break. To minimize the blast radius of these breakages, handle errors correctly. Yes, critical errors can crash the system but first try to articulate why a specific error should be critical. The more clearly you can articulate what the error is and how it should be handled, the more robust your code is. As an added benefit you will have a fraction of the operational load.

Software is Human

You make design decisions and implementation choices on a regular basis. Your digital fingerprint will be on the software. This ownership excites me and I strive to deliver robust, well defined solutions for both the good of the product and the satisfaction of the stakeholders.